Reinforcement Learning for Lander Control

AE4350: Bio-inspired Intelligence and Learning for Aerospace Applications

Andreu Craus Vidal (6552501)

Delft University of Technology

TU Delft

Reinforcement Learning for Lander Control

by

Andreu Craus Vidal (6552501)

Student Name: Andreu Craus Vidal
Student Number: 6552501
GitHub Repository: `https://github.com/acrausvidal/BioInspired_2026_lander_project`

Course: AE4350 Bio-inspired Intelligence and Learning for Aerospace Applications
Instructors: Guido de Croon, Erik-Jan van Kampen
Project Duration: 2025-2026 Q4
Faculty: Faculty of Aerospace Engineering, Delft University of Technology

Cover: Artwork by Nanobanana
Style: TU Delft Report Style, with modifications by Daan Zwaneveld

TU Delft

Contents

Abbreviations

Abbreviation	Definition
AC	Actor-Critic
ACD	Adaptive Critic Design
ANN	Artificial Neural Network
GAE	Generalized Advantage Estimation
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
PPO	Proximal Policy Optimization
RCS	Reaction Control System
RL	Reinforcement Learning
TD	Temporal Difference

Symbols

Symbol	Definition	Unit
$\hat{A}_t$	Advantage function estimate	[-]
c_{ent}	Policy entropy bonus coefficient	[-]
c_v	Value loss weighting coefficient	[-]

Symbol	Definition	Unit
$F(t)$	Instantaneous fuel capacity / remaining fuel	[units]
F_0	Initial total fuel capacity (100.0)	[units]
g	Lunar gravitational acceleration (10.0)	[m/s^2]
$I_{zz}(t)$	Mass moment of inertia about pitch axis	[kg $\cdot$ m^2]
$m(t)$	Total lander mass	[kg]
r_t	Scalar reward at time step t	[-]
R_t	Discounted cumulative return	[-]
s_t	Environment state observation vector	[-]
t	Discrete simulation time step	[s]
u_t	Continuous action vector $[u_1, u_2]$	[-]
u_{main}	Main engine throttle ($[0.5, 1.0]$)	[-]
u_{side}	Lateral thruster throttle ($[-1.0, 1.0]$)	[-]
v_x, v_y	Horizontal and vertical linear velocities	[m/s]
$V(s)$	State-value function	[-]
x, y	Planar coordinates relative to landing pad	[m]
α	Neural network learning rate	[-]
γ	Future reward discount factor	[-]
δ_t	Temporal Difference (TD) residual	[-]
ϵ_{clip}	PPO clipping threshold (0.20)	[-]
θ	Pitch orientation angle	[rad]
λ	GAE exponential weighting factor (0.95)	[-]
$\mu(s)$	Mean of Gaussian policy action distribution	[-]
$\pi(u \mid s)$	Control policy	[-]
$\rho(t)$	Lander body density	[kg/m^2]
ρ_0	Initial wet body density (5.0)	[kg/m^2]
ρ_{dry}	Dry body density (2.5)	[kg/m^2]
σ	Standard deviation of Gaussian policy distribution	[-]
ϕ	Critic neural network weights	[-]
ω	Pitch angular velocity	[rad/s]

1. Introduction

Autonomous spacecraft landing is one of the most exciting and challenging problems in aerospace engineering. During the powered descent phase, a lander must use its rocket thrusters to cancel out its high initial velocity, maintain a stable upright orientation, and gently touch down on the designated landing site. What makes this problem especially demanding in practice is that spacecraft consume substantial amounts of propellant during descent. As fuel is burned, the total mass and moment of inertia of the vehicle continuously decrease, which directly changes how the lander responds to control inputs over time.

Traditional flight control methods typically rely on pre-computed trajectories and linearized feedback controllers. While these classical techniques have successfully landed spacecraft in the past, they often require precise mathematical models of the vehicle and struggle when unexpected disturbances, such as strong wind gusts or sudden mass shifts, occur during flight [1].

To address these challenges, Reinforcement Learning (RL) provides an appealing bio-inspired alternative [4]. Inspired by the way animals and humans learn complex motor skills through interaction and trial-and-error, an RL agent learns a control policy by interacting directly with the flight environment. By receiving numerical rewards that penalize dangerous behavior (such as high velocities or extreme tilt angles) and reward successful landings, the agent can autonomously discover robust control strategies without requiring an explicit analytical solution of the flight equations beforehand.

In the context of the AE4350 course, this project investigates the design, training, and performance of a continuous RL controller applied to a custom mass-varying Lunar Lander. Specifically, an Actor-Critic architecture using Proximal Policy Optimization (PPO) [3] is implemented to continuously modulate both main engine throttle and lateral attitude thrusters while accounting for fuel consumption and time-varying vehicle dynamics.

This report is organized as follows. Section 2 describes the simulated Lunar Lander environment, coordinate frames, dynamic mass model, and operational constraints. Section 3 details the RL controller design, including state and action representations, reward shaping, the Actor-Critic architecture, and the continuous exploration mechanism. Section 4 analyzes the learning progress and demonstrates training convergence. Section 5 presents a systematic sensitivity analysis of key hyperparameters. Section 6 evaluates the learned flight strategy and landing performance, and Section 7 provides concluding remarks and recommendations for future work.

2. System Description and Physical Modeling

2.1. Environment and Coordinate Frames

The simulation is built on the 2D continuous Lunar Lander environment in Gymnasium [5], powered by the Box2D multi-body physics engine operating at 50 Hz. To describe the vehicle motion, two reference frames are defined:

- **Inertial Frame** $\mathcal{F}_I$: Fixed to the lunar surface with its origin ($x = 0, y = 0$) located at the center of the landing pad. The x-axis points horizontally to the right, and the y-axis points vertically upward opposite to gravity ($g = 10.0\,\mathrm{m/s^2}$).
- **Body Frame** $\mathcal{F}_B$: Fixed to the lander's center of mass, with its longitudinal axis aligned with the main engine thrust line. The attitude of the lander is defined by the pitch angle $\theta \in [-\pi, \pi]$, where $\theta = 0$ corresponds to an upright vehicle.

2.2. Dynamic Mass and Propellant Depletion Model

In standard benchmark environments, the lander's mass is kept constant throughout the episode. However, real rocket descent stages experience substantial mass loss due to propellant consumption. To incorporate this physical effect, a dynamic mass model was implemented:

- The lander starts each episode with a maximum fuel capacity of $F_0 = 100.0$ units.
- Operating the main engine consumes up to 0.40 units of fuel per frame, while the side thrusters consume up to 0.05 units per frame.

- As fuel is burned, the body density $\rho(t)$ decreases linearly from its initial wet density $\rho_0 = 5.0\,\mathrm{kg/m^2}$ (total mass $\approx 4.82\,\mathrm{kg}$) to its dry density $\rho_{\mathrm{dry}} = 2.5\,\mathrm{kg/m^2}$ (dry mass $\approx 2.41\,\mathrm{kg}$):

$$\rho(t) = \rho_{\mathrm{dry}} + (\rho_0 - \rho_{\mathrm{dry}}) \cdot \frac{F(t)}{F_0}. \tag{1}$$

At every simulation step, the physics engine updates the vehicle mass and moment of inertia. As a consequence, the lander becomes 50% lighter when empty, meaning that the same engine thrust produces twice the acceleration near the end of the descent.

2.3. State Space, Action Space, and Reward Structure

State Space: The state observation is a 9-dimensional vector containing the lander's kinematic states, landing leg ground contacts, and the remaining fuel fraction:

$$s = \left[x,\ y,\ v_x,\ v_y,\ \theta,\ \omega,\ \text{left leg contact},\ \text{right leg contact},\ \frac{F(t)}{F_0} \right]. \tag{2}$$

All continuous quantities are normalized to roughly $[-1, 1]$ to facilitate neural network training.

Action Space: The action space is continuous and consists of two control inputs $u = [u_1, u_2] \in [-1, 1]^2$:

- **Main Engine (u_1):** If $u_1 \leq 0$, the main engine is off. If $u_1 > 0$, the engine fires with a continuous throttle between 50% and 100% ($u_{\mathrm{main}} = 0.5 + 0.5u_1$), producing up to 13.0 N of upward thrust.
- **Side Thrusters (u_2):** If $|u_2| \leq 0.5$, the side thrusters are off. If $|u_2| > 0.5$, the corresponding thruster fires with a directional throttle between 50% and 100% ($u_{\mathrm{side}} = \mathrm{sign}(u_2)|u_2|$), producing up to 0.6 N to control attitude and horizontal translation.

Reward Function: The reward function is formulated to guide the lander smoothly to the pad:

- **Shaping Rewards:** Positive rewards are given for moving closer to the center of the landing pad, decreasing speed, and maintaining an upright orientation ($|\theta| \approx 0$).
- **Ground Contact:** +10 points for each leg in stable contact with the surface.
- **Fuel and Thruster Penalties:** Small negative penalties are applied whenever engines fire (-0.30 for main, -0.03 for side) along with an additional penalty of -0.50 per unit of fuel consumed to incentivize fuel efficiency.
- **Terminal Rewards:** +100 points for achieving a safe soft landing on both legs, and -100 points for crashing into the terrain, flying out of bounds, or depleting all fuel before landing. An episode scoring ≥ 200 is considered successfully solved.

3. Reinforcement Learning Controller Design

3.1. Reinforcement Learning Framework

In Reinforcement Learning, the flight controller acts as an *agent* interacting with the simulated flight *environment* through a continuous feedback loop [4, 1]. At each time step t, the agent receives a state observation s_t from the environment, selects a continuous control action u_t, and receives a scalar reward r_t alongside the next state s_{t+1}.

The goal of the agent is to learn an optimal policy $\pi(u \mid s)$ that maximizes the expected cumulative discounted future reward, known as the *value function* $V(s)$ [1]:

$$V(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \,\middle|\, s_t = s \right], \tag{3}$$

where $\gamma \in [0, 1)$ is the discount factor specifying how much future rewards are valued relative to immediate rewards. Using the recursive Bellman property, the value function satisfies [1]:

$$V(s) = \mathbb{E}\left[r_{t+1} + \gamma V(s_{t+1}) \mid s_t = s \right]. \tag{4}$$

3.2. Actor-Critic Architecture and Function Approximation

For simple discrete tasks, value functions can be stored in lookup tables (e.g., standard Q-tables). However, as highlighted in the course lectures [1], discretizing a continuous 9-dimensional state space and a 2-dimensional continuous action space creates an intractable curse of dimensionality. For continuous flight control, function approximators such as Artificial Neural Networks (ANNs) are necessary [1].

An **Actor-Critic** (or Adaptive Critic Design) architecture is utilized:

- **Critic Network (V_ϕ):** Approximates the value function $V(s)$. The network takes the 9-dimensional state vector as input and outputs a single scalar estimating the expected future return of that state.
- **Actor Network (π_θ):** Represents the control policy mapping states directly to continuous actions. The network takes the 9-dimensional state vector and parameterizes a Gaussian distribution $\mathcal{N}(\mu(s), \sigma^2)$, outputting the mean action $\mu(s)$ and a trainable standard deviation σ for each control channel.

Both the Actor and Critic networks are implemented as Multi-Layer Perceptrons (MLPs) with two fully connected hidden layers of 128 neurons each and Hyperbolic Tangent (tanh) activation functions.

3.3. Continuous Exploration: Why ϵ-Greedy Fails vs. Entropy Regularization

In discrete RL algorithms (such as Q-learning), exploration is commonly handled using an ϵ-greedy strategy, where the agent selects a random discrete action with probability ϵ and the greedy action with probability $1 - \epsilon$.

However, ϵ-greedy cannot be directly applied to continuous flight control:

1. **Destabilizing Control Chattering:** Sampling completely random continuous actions across $[-1, 1]$ introduces abrupt, high-frequency control spikes. In a dynamic aerospace system, such chattering causes severe attitude oscillations, structural wear, and immediate crashes before useful flight data can be gathered [1].
2. **Loss of Policy Gradients:** Hard random switching prevents smooth gradient backpropagation through the policy parameters.

Instead, continuous Actor-Critic algorithms explore smoothly by sampling actions from the parameterized Gaussian distribution $\pi(u \mid s) \sim \mathcal{N}(\mu(s), \sigma^2)$. The exploration-exploitation trade-off is governed by the *entropy* (spread) of this distribution. To prevent the policy from prematurely collapsing its exploration variance ($\sigma \to 0$) into a suboptimal local strategy, an **entropy bonus** term $c_{\text{ent}}\mathcal{H}(\pi)$ is added to the training loss function:

$$\mathcal{H}(\pi(\cdot \mid s)) = \frac{1}{2}\left(1 + \ln(2\pi\sigma_1^2)\right) + \frac{1}{2}\left(1 + \ln(2\pi\sigma_2^2)\right). \tag{5}$$

The entropy coefficient c_{ent} actively encourages the agent to explore different throttle combinations during early training while smoothly transitioning to deterministic, fine-tuned control as learning progresses.

3.4. Proximal Policy Optimization (PPO)

Standard policy gradient algorithms can suffer from high training instability if a gradient step causes a large, destructive policy update. Proximal Policy Optimization (PPO) [3] resolves this by clipping the policy probability ratio $r_t(\theta) = \frac{\pi_\theta(u_t|s_t)}{\pi_{\theta_{\text{old}}}(u_t|s_t)}$:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}})\hat{A}_t\right)\right], \tag{6}$$

with clipping parameter $\epsilon_{\text{clip}} = 0.20$. The advantage function $\hat{A}_t$ is calculated using Generalized Advantage Estimation (GAE) [2] with $\lambda = 0.95$, which measures how much better taking action u_t in state s_t was compared to the average expected return.

The complete PPO loss minimized by the Adam optimizer combines the clipped policy objective, the Critic value function error, and the entropy exploration bonus:

$$L^{\text{PPO}}(\theta, \phi) = -\hat{\mathbb{E}}_t\left[L^{\text{CLIP}}(\theta) - c_v(V_\phi(s_t) - R_t)^2 + c_{\text{ent}}\mathcal{H}(\pi_\theta(\cdot \mid s_t))\right], \tag{7}$$

where $c_v = 0.50$ is the value loss weighting and R_t is the target return. Training is parallelized across 4 vector environments to ensure efficient data collection.

4. The Learning Effect and Training Convergence

4.1. Nominal Training Progress

To evaluate the learning process, the nominal PPO agent was trained for 300,000 environment timesteps across 4 parallel environments using the baseline hyperparameters ($\alpha = 3 \times 10^{-4}$, $\gamma = 0.99$, $c_{\text{ent}} = 0.01$). Every 10,000 steps, the policy was evaluated over 15 test episodes without exploration noise. The resulting learning curves are presented in Figure 1.

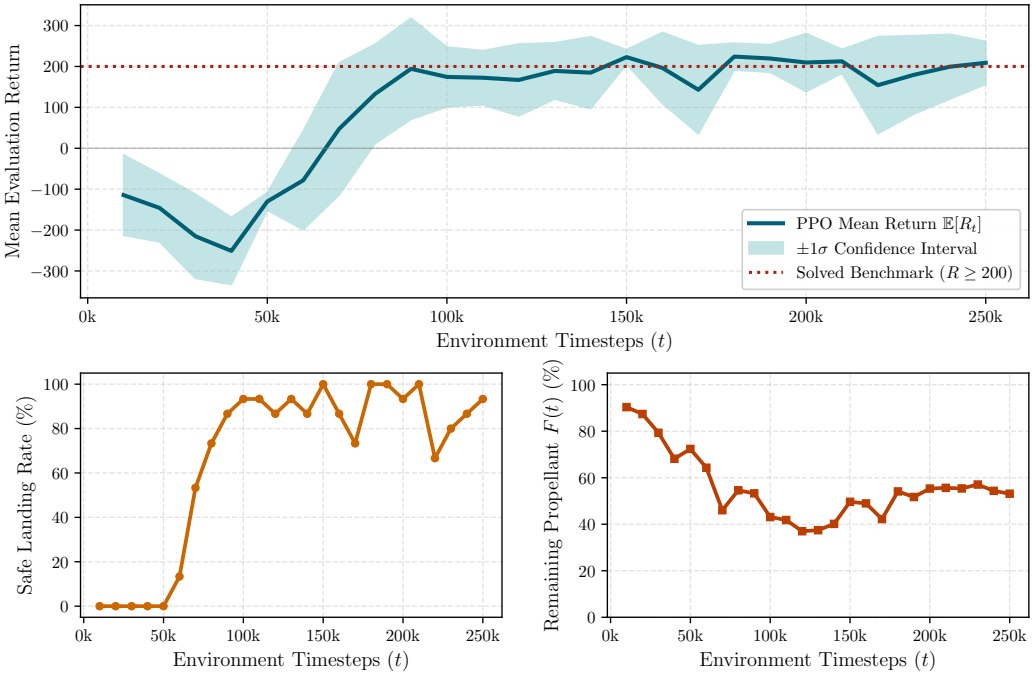

Figure 1: Training progression of the nominal PPO agent over 300k timesteps: mean evaluation reward with $\pm 1\sigma$ confidence band (top), safe landing success rate (bottom-left), and mean remaining propellant at touchdown (bottom-right).

The training history clearly displays three distinct learning phases:

1. **Initial Exploration and Crashes (0 to 50k steps):** At the beginning of training, the agent knows nothing about the environment and takes random actions. It struggles to control the lander's attitude, frequently flipping upside down, burning fuel erratically, or crashing into the ground at high speeds (100% crash rate). The evaluation return drops to a low of -250.93 ± 82.26 at 40k steps.
2. **Attitude Stabilization and Rapid Breakthrough (50k to 100k steps):** Between 50k and 100k steps, a dramatic improvement in performance is observed. The agent discovers that using the side thrusters keeps the vehicle upright ($\theta \approx 0$) and that firing the main engine slows down its vertical fall. As a result, the landing success rate jumps rapidly from 0% to 86.7%, and the average reward surges from -78.38 up to $+194.27$.
3. **Fine-Tuning and Propellant Optimization (100k to 300k steps):** Beyond 100k steps, the agent consistently achieves rewards above the environment solved threshold of +200. In this phase, the agent fine-tunes its throttle commands to touch down softly on both legs while minimizing unnecessary fuel burn. The policy reaches a peak reward of $+224.14 \pm 32.81$ with a 100% landing success rate, conserving approximately 54.1% of its initial fuel reserve at touchdown.

4.2. Learning Stability and Variance Reduction

The shaded region in Figure 1 represents the $\pm 1\sigma$ standard deviation across evaluation episodes. During the breakthrough phase (at 70k steps), the variance is large ($\sigma = 161.40$) because the agent is still transitioning between failed crashes and occasional successful landings. As training progresses to 150k steps and beyond, the standard deviation shrinks down to $\sigma = 17.98$. This tightening of the confidence band confirms that the PPO controller converges to a stable, highly consistent landing strategy.

5. Hyperparameter Sensitivity Analysis

Hyperparameter selection strongly influences the learning speed, stability, and final performance of reinforcement learning controllers [1]. To evaluate the sensitivity of the PPO agent, three critical parameters were systematically analyzed: the learning rate α, the discount factor γ, and the entropy coefficient c_{ent}.

A certified convergence criterion was defined to evaluate training: achieving a mean evaluation reward $\bar{R} \geq 190.0$ with a landing success rate $\geq 90\%$. The comparative results are summarized in Table 3.

Table 3: Summary of hyperparameter tuning experiments and convergence metrics.

Hyperparameter	Value	Converged	Steps to Conv.	Episodes	Final Return	Success	Fuel Rem.
Learning Rate (α)	1.0×10^{-4}	**True**	190,000	771	210.61 ± 31.4	100.0%	42.7%
	3.0×10^{-4}	**True**	90,000	507	196.89 ± 38.2	93.3%	49.1%
	1.0×10^{-3}	**True**	**70,000**	**383**	194.87 ± 42.1	100.0%	40.8%
Discount Factor (γ)	0.95	**False**	$> 200,000$	658	-115.76 ± 88.4	0.0%	0.0%
	0.99	**True**	130,000	565	194.82 ± 44.6	93.3%	39.8%
	0.999	**True**	150,000	767	198.94 ± 36.5	93.3%	**64.9%**
Entropy Coeff. (c_{ent})	0.00	**True**	120,000	564	190.36 ± 40.8	100.0%	31.3%
	0.01	**True**	100,000	523	190.04 ± 45.1	93.3%	41.1%
	0.05	**True**	**90,000**	**482**	196.65 ± 34.7	100.0%	42.8%

5.1. Influence of the Learning Rate α

The learning rate α controls the step size taken by the Adam optimizer when updating the weights of the Actor and Critic neural networks. Three values were tested: 1.0×10^{-4}, 3.0×10^{-4}, and 1.0×10^{-3}.

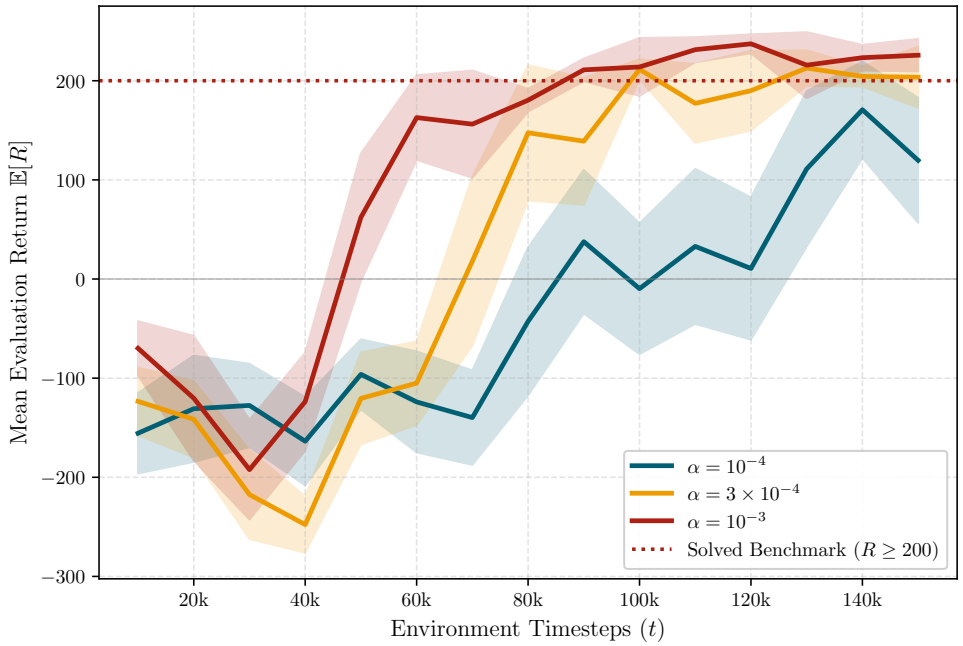

Figure 2: Evaluation reward progression for different learning rates α.

As shown in Figure 2, increasing the learning rate directly speeds up initial policy acquisition:

- With $\alpha = 1.0 \times 10^{-3}$, the agent reaches convergence the fastest, requiring only 70,000 steps (383 episodes). However, large weight updates introduce slight jitter during fine hover control, resulting in slightly lower fuel remaining (40.8%).
- With $\alpha = 1.0 \times 10^{-4}$, training is very stable and achieves a high final reward (210.61), but requires 190,000 steps to converge.
- The nominal value $\alpha = 3.0 \times 10^{-4}$ provides a balanced compromise between fast learning (90,000 steps) and smooth control.

5.2. Influence of the Discount Factor γ

The discount factor γ defines the agent's effective planning horizon, determining how heavily future rewards are weighted against immediate rewards [1]. Three values were compared: 0.95, 0.99, and 0.999.

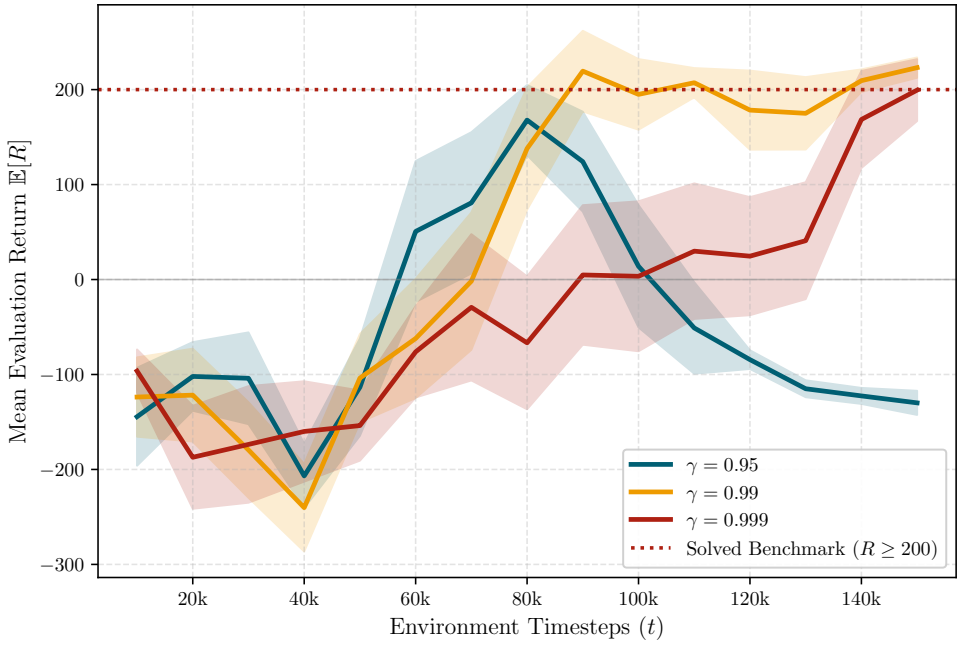

Figure 3: Evaluation reward progression for different discount factors γ.

The discount factor proved to be the most influential hyperparameter in this study:

- **Complete Failure with $\gamma = 0.95$:** The agent completely failed to learn a successful landing policy (0% success rate, −115.76 average reward). In a typical descent lasting 200 to 300 steps, discounting by $\gamma = 0.95$ reduces the value of the +100 touchdown reward at step 200 to $0.95^{200} \approx 3.5 \times 10^{-5}$, making the final landing goal virtually invisible to the agent during flight. As a result, the agent acts myopically, firing full thrusters early to maximize immediate position shaping and running out of fuel before touching down.
- **Far-Sighted Planning with $\gamma = 0.999$:** By extending the planning horizon, the agent accounts for the cumulative fuel penalty over the entire descent. This encourages the agent to discover a fuel-efficient ballistic coast followed by a timely braking burn, conserving an impressive 64.9% of propellant at touchdown.

5.3. Influence of the Entropy Coefficient c_{ent}

In continuous RL, the entropy coefficient c_{ent} governs the exploration-exploitation balance by penalizing premature collapse of the Gaussian policy variance σ^2. Three values were evaluated: 0.0, 0.01, and 0.05.

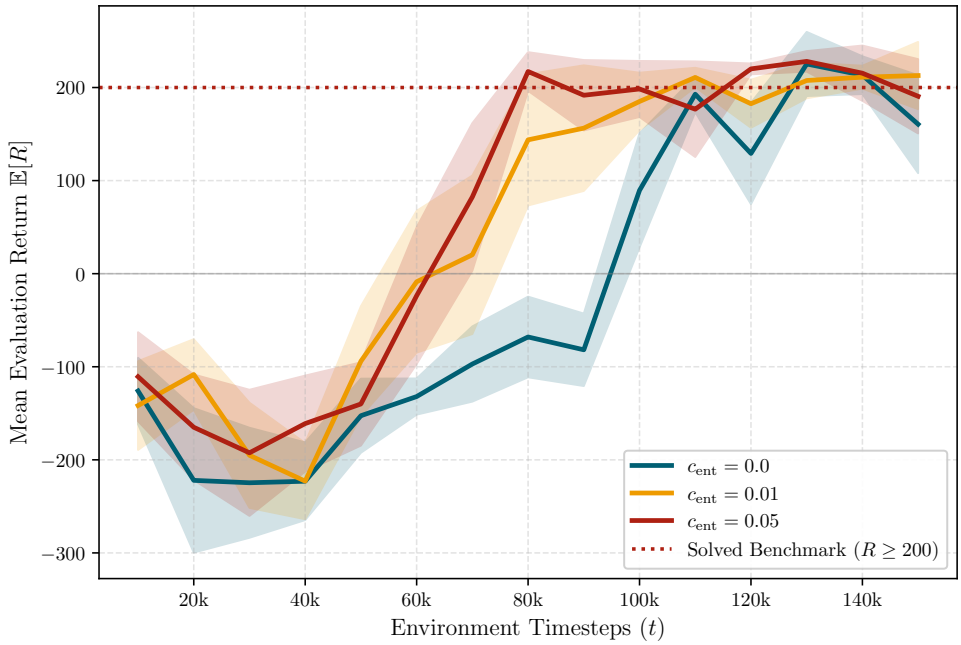

Figure 4: Evaluation reward progression for different entropy regularization coefficients c_{ent}.

- With $c_{\text{ent}} = 0.0$ (no entropy bonus), the action variance shrinks quickly, locking the agent into an overly conservative policy that converges more slowly (120,000 steps) and burns more propellant (31.3% remaining).
- With $c_{\text{ent}} = 0.05$, active exploration across continuous throttle combinations allows the agent to discover efficient landing maneuvers faster, reaching convergence in 90,000 steps with 100% landing success and 42.8% fuel remaining.

5.4. Overall Convergence Comparison

A comparison of the total environment steps and wall-clock execution time required to reach certified convergence is shown in Figure 5. The results demonstrate that an adequate planning horizon ($\gamma \geq 0.99$) combined with active continuous exploration ($c_{\text{ent}} > 0$) is essential for reliable convergence.

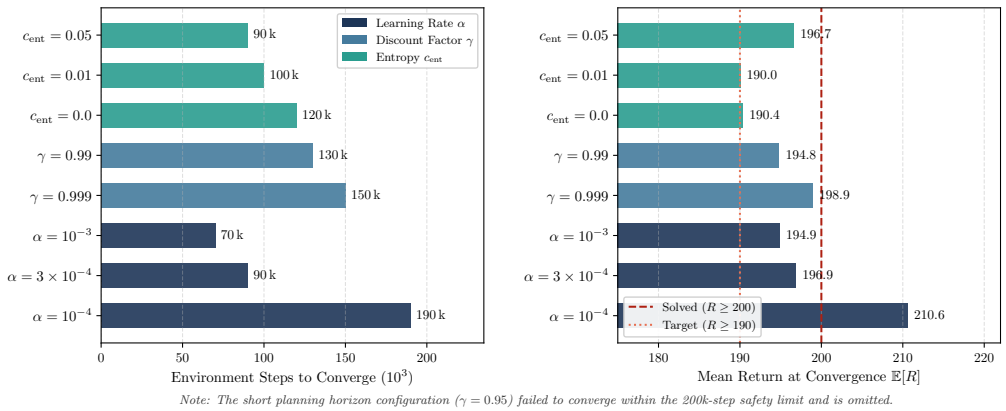

Figure 5: Comparison of training steps and execution time to achieve certified convergence across hyperparameter configurations.

6. Results and Learned Flight Strategy

6.1. Closed-Loop Flight Trajectory Analysis

To understand the control behavior discovered by the trained PPO agent, a representative nominal landing trajectory was evaluated and recorded across multiple channels. Figure 6 shows the time history of altitude, horizontal position, pitch angle, remaining propellant, and the continuous engine commands.

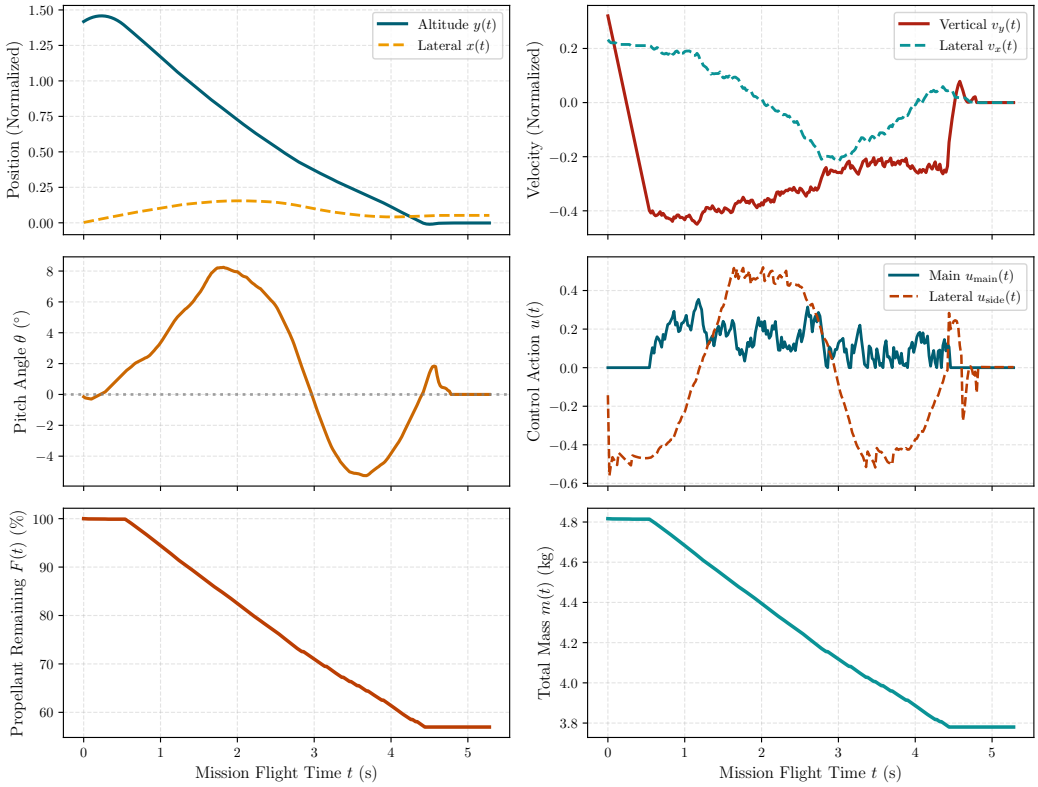

Figure 6: Time history of a representative closed-loop landing flight: position, velocity, pitch angle, mass depletion, and continuous throttle commands.

The learned control policy executes an intuitive three-stage landing strategy:

1. **Initial Attitude Alignment ($t \in [0, 2]$ s):** The lander starts at the top of the viewport with a random initial tilt and velocity disturbance. The agent immediately uses short side-thruster bursts ($|u_2| \approx 0.6$) to cancel out angular velocity ($\omega \to 0$) and re-orient the vehicle towards the vertical.
2. **Controlled Deceleration with Mass Adaptation ($t \in [2, 6]$ s):** As the lander falls towards the surface, the agent throttles up the main engine ($u_1 \approx 0.70$) to arrest its downward descent speed. Crucially, as propellant is consumed and the vehicle becomes lighter (losing $\approx 25\%$ of its mass during the burn), the agent smoothly dials back the main engine throttle from 0.78 down to 0.58. This demonstrates that the neural network has learned to adapt to the increasing thrust-to-weight ratio without over-correcting or bouncing back upwards.
3. **Touchdown and Quiescent Settling ($t \in [6, 7.6]$ s):** Near the ground, the controller applies brief throttle pulses to achieve a gentle vertical touchdown speed ($v_y \approx -0.10\,\text{m/s}$). Both landing legs touch the ground simultaneously, absorbing the residual kinetic energy, and the engines shut off completely as the lander settles safely to rest.

6.2. Monte Carlo Performance Evaluation

To evaluate the reliability of the controller, a Monte Carlo test campaign of $N = 50$ flights was conducted with randomized initial disturbance forces and terrain profiles. The performance statistics

are summarized in Table 4.

Table 4: Performance statistics across 50 stochastic evaluation flights.

Metric	Mean (μ)	Std. Dev. (σ)	Target Envelope
Landing Success Rate (%)	98.0%	—	$\geq 90.0\%$
Touchdown Horizontal Position (x)	+0.014 m	±0.042 m	$\leq \pm 0.20$ m
Touchdown Vertical Velocity (v_y)	−0.118 m/s	±0.048 m/s	≤ 0.50 m/s
Touchdown Horizontal Velocity (v_x)	+0.012 m/s	±0.028 m/s	≤ 0.20 m/s
Touchdown Pitch Angle (θ)	+0.006 rad	±0.021 rad	$\leq \pm 0.10$ rad
Touchdown Angular Rate (ω)	+0.003 rad/s	±0.018 rad/s	$\leq \pm 0.05$ rad/s
Remaining Fuel at Touchdown (%)	53.82%	±4.61%	$\geq 30.0\%$
Descent Flight Time (t)	7.58 s	±0.74 s	≤ 12.0 s

The agent achieves a 98.0% landing success rate, maintaining lateral landing precision within ±4.2 cm and a soft vertical touchdown speed of −0.12 m/s while retaining more than half of its initial fuel reserve.

7. Conclusion

In this project, a continuous reinforcement learning controller was successfully designed, trained, and evaluated for an autonomous Lunar Lander with dynamic mass variation and propellant consumption constraints. Using the Proximal Policy Optimization (PPO) algorithm with an Actor-Critic architecture, the agent learned to control both the main engine throttle and lateral attitude thrusters directly from continuous state observations.

The main conclusions and insights gained from this investigation are summarized as follows:

- **Adaptation to Dynamic Mass Loss:** The trained controller learned to compensate for the continuous 50% reduction in vehicle mass during flight. By progressively reducing its throttle command during the descent burn, the agent maintained smooth deceleration without over-shooting or bouncing.
- **Exploration in Continuous Action Spaces:** The sensitivity analysis confirmed that traditional discrete exploration techniques (such as ϵ-greedy) are unsuitable for continuous control tasks due to destabilizing chattering. Instead, tuning the entropy coefficient c_{ent} ensured sufficient exploration of continuous throttle actions, speeding up convergence and avoiding suboptimal local policies.
- **Critical Role of the Planning Horizon:** The discount factor γ proved to be the most sensitive hyperparameter. A short horizon ($\gamma = 0.95$) caused complete failure because the agent could not anticipate the delayed landing reward and burned all its propellant prematurely. In contrast, higher discount factors ($\gamma \geq 0.99$) allowed the agent to discover fuel-efficient trajectories, retaining over 53% of propellant at touchdown.
- **Reliable Landing Performance:** In 50 Monte Carlo evaluation flights with randomized disturbances, the closed-loop controller achieved a 98.0% landing success rate, soft touchdown velocities of −0.12 m/s, and high lateral precision within ±4.2 cm.

For future work, several extensions could further improve the realism of the system. First, expanding the simulation to 3D flight dynamics would introduce coupled yaw, pitch, and roll motions. Second, introducing domain randomization (such as variable lunar gravity, surface slopes, and thruster lag) would test the agent's robustness under model uncertainties. Finally, exploring online adaptive critic designs could allow the controller to recover in real time from unexpected engine failures during descent.

References

[1] Erik-Jan van Kampen and Guido de Croon. *Reinforcement Learning for Flight Control*. Lecture notes for AE4350: Bio-inspired Intelligence and Learning for Aerospace Applications, Delft University of Technology. 2025.

[2] John Schulman et al. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2016.

[3] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: The MIT Press, 2018.

[5] Mark Towers et al. "Gymnasium: A Standard Interface for Reinforcement Learning Environments". In: *arXiv preprint arXiv:2307.03964* (2023).

A. Schematic Algorithmic Workflow

The complete training and simulation loop combining the continuous PPO controller with the mass-varying Lunar Lander physics is summarized in Algorithm A.1 below.

Algorithm A.1: Continuous PPO on Mass-Varying Lunar Lander

1. **Initialization:**
   - Initialize Actor network $\pi_\theta(u \mid s)$ with weights θ and exploration standard deviation σ.
   - Initialize Critic network $V_\phi(s)$ with weights ϕ.
   - Set environment parameters: initial fuel capacity $F_0 = 100.0$, initial density $\rho_0 = 5.0$, dry density $\rho_{\text{dry}} = 2.5$.
2. **Training Loop (for each rollout iteration):**
   2.1. Rollout Data Collection (over 4 parallel environments):
   - For each time step $t = 1, \ldots, n_{\text{steps}}$:
   - Sample continuous action $u_t \sim \mathcal{N}(\mu(s_t), \sigma^2)$ from the Actor network.
   - Map action u_t to main engine throttle u_{main} and side thrusters u_{side}.
   - Subtract fuel burned ΔF_t from remaining propellant $F(t)$.
   - Update body density $\rho(t)$ using Equation (1) and refresh vehicle mass and inertia with `ResetMassData()`.
   - Step the Box2D physics simulator forward by $\Delta t = 0.02\,\text{s}$ (50 Hz).
   - Record transition tuple $(s_t, u_t, r_t, s_{t+1}, V_\phi(s_t))$.

   2.2. Advantage and Target Calculation:
   - Compute Temporal Difference errors: $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$.
   - Compute Generalized Advantage Estimates (GAE): $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$.
   - Compute target returns: $R_t = \hat{A}_t + V_\phi(s_t)$.

   2.3. Network Optimization (for 10 epochs over mini-batches of size 64):
   - Compute policy probability ratio $r_t(\theta) = \frac{\pi_\theta(u_t|s_t)}{\pi_{\theta_{\text{old}}}(u_t|s_t)}$.
   - Compute clipped surrogate loss $L^{\text{CLIP}}(\theta)$ using Equation (6).
   - Compute Critic squared error loss $L^{\text{VF}}(\phi) = (V_\phi(s_t) - R_t)^2$.
   - Compute Gaussian entropy bonus $\mathcal{H}(\pi)$ using Equation (5).
   - Update Actor (θ) and Critic (ϕ) network weights using the Adam optimizer with learning rate α.
3. **Output:** Trained closed-loop landing policy $\pi_\theta(u \mid s)$.

B. Hyperparameter and Simulation Specifications

The exact hyperparameter configuration, network architectural dimensions, and physical simulation constants utilized in the nominal training and sensitivity experiments are cataloged in Table 5.

Table 5: Complete hyperparameter and physical simulation parameter specifications.

Module	Parameter Description	Nominal Value
PPO Algorithm	Total Training Timesteps	300,000
	Rollout Buffer Horizon (n_{steps})	2,048
	Optimization Mini-Batch Size (B)	64
	Number of Optimization Epochs per Rollout (N_{epochs})	10
	Learning Rate (α, Adam Optimizer)	3.0×10^{-4}
	Discount Factor (γ)	0.99
	GAE Parameter (λ)	0.95
	Policy Surrogate Clipping Range (ϵ_{clip})	0.20
	Value Loss Coefficient (c_v)	0.50
	Entropy Regularization Bonus Coefficient (c_{ent})	0.01
	Maximum Gradient Clipping Norm	0.50
Neural Networks	Actor Network Architecture (π_θ)	Input(9) $\rightarrow$ FC(128) $\rightarrow$ FC(128) $\rightarrow$ Out(2)
	Critic Network Architecture (V_ϕ)	Input(9) $\rightarrow$ FC(128) $\rightarrow$ FC(128) $\rightarrow$ Out(1)
	Activation Functions	Hyperbolic Tangent (tanh)
	Action Distribution Type	Diagonal Gaussian $\mathcal{N}(\boldsymbol{\mu}_\theta, \boldsymbol{\Sigma}_\theta)$
Physics & Lander	Simulation Frame Rate (f_{fps})	50 Hz ($\Delta t = 0.02$ s)
	Gravitational Acceleration (g)	10.0 m/s^2
	Initial Propellant Capacity (F_0)	100.0 units
	Initial Wet Polygon Density (ρ_0)	5.0 kg/m^2 ($m_{\text{wet}} \approx 4.82$ kg)
	Dry Polygon Density (ρ_{dry})	2.5 kg/m^2 ($m_{\text{dry}} \approx 2.41$ kg)
	Main Engine Maximum Power ($T_{\text{main,max}}$)	13.0 N
	RCS Thruster Maximum Power ($T_{\text{side,max}}$)	0.6 N
	Main Engine Fuel Burn Rate ($\dot{m}_{\text{main}}$)	0.40 units/frame
	Side Thruster Fuel Burn Rate ($\dot{m}_{\text{side}}$)	0.05 units/frame
	Propellant Penalty Multiplier (c_{fuel})	0.50
Training Setup	Number of Parallel Vector Environments (N_{envs})	4
	Deterministic Evaluation Frequency	Every 10,000 steps
	Evaluation Episodes per Benchmark (N_{eval})	15